

OSSP (Operating Systems And Systems Programming) - 25CS2104E

Roll No. : 2520030029

Name: Pujith Pavan Kumar

SEC-8

PRACTICAL-1:

Aim: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

Using Linux terminal commands (uname, lscpu, lsblk, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

```
siddinapujith07@DESKTOP-FDHCAIT:~$ mkdir OSSP
mkdir: OSSP: File exists
siddinapujith07@DESKTOP-FDHCAIT:~$ cd OSSP
siddinapujith07@DESKTOP-FDHCAIT:~/OSSP$ nano command_exec.c
siddinapujith07@DESKTOP-FDHCAIT:~/OSSP$ ls
OSSP  command_exec  command_exec.c
siddinapujith07@DESKTOP-FDHCAIT:~/OSSP$ ./command_exec
Enter a Linux command: uname

Parent Process
Parent PID: 863

Child Process
Child PID: 866
Parent PID: 863
Linux
Child process completed.
siddinapujith07@DESKTOP-FDHCAIT:~/OSSP$ ./command_exec
Enter a Linux command: lscpu

Parent Process
Parent PID: 876

Child Process
Child PID: 877
Parent PID: 876
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          39 bits physical, 48 bits virtual
Byte Order:             Little Endian
CPU(s):                 8
On-line CPU(s) list:    0-7
Vendor ID:              GenuineIntel
```

```

Vendor ID: GenuineIntel
Model name: 11th Gen Intel(R) Core(TM) i7-1185G7 @ 3.00GHz
CPU family: 6
Model: 140
Thread(s) per core: 2
Core(s) per socket: 4
Socket(s): 1
Stepping: 1
BogoMIPS: 5990.42
Flags: fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ss ht
syscall nx pdpe1gb rdtscp lm constant_tsc arch_perfmon rep_good nopl xtopology tsc_reliable nonstop_tsc
cpuid tsc_known_freq pni pclmulqdq vmx ssse3 fma cx16 pdcm pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_de
adline_timer aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch ssbd ibrs ibpb stibp ibrs_en
hanced tpr_shadow ept vpid ept_ad fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid avx512f avx512dq
rdseed adx smap avx512ifma clflushopt clwb avx512cd sha_ni avx512bw avx512vl xsaveopt xsavec xgetbv1 xsa
ves vnmi avx512vbmi umip avx512_vbmi2 gfni vaes vpclmulqdq avx512_vnni avx512_bitalg avx512_vpoperntdqd rd
pid movdiri movdir64b fsrm avx512_vp2intersect md_clear ibt flush_lld arch_capabilities

Virtualization features:
Virtualization: VT-x
Hypervisor vendor: Microsoft
Virtualization type: full
Caches (sum of all):
L1d: 192 KiB (4 instances)
L1i: 128 KiB (4 instances)
L2: 5 MiB (4 instances)
L3: 12 MiB (1 instance)
NUMA:
NUMA node(s): 1
NUMA node0 CPU(s): 0-7
Vulnerabilities:
Gather data sampling: Unknown: Dependent on hypervisor status
Ghostwrite: Not affected
Indirect target selection: Mitigation; Aligned branch/return thunks

```

```

Indirect target selection: Mitigation; Aligned branch/return thunks
Itlb multihit: Not affected
L1tf: Not affected
Mds: Not affected
Meltdown: Not affected
Mmio stale data: Not affected
Old microcode: Not affected
Reg file data sampling: Not affected
Retbleed: Mitigation; Enhanced IBRS
Spec rstack overflow: Not affected
Spec store bypass: Mitigation; Speculative Store Bypass disabled via prctl
Spectre v1: Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Spectre v2: Mitigation; Enhanced / Automatic IBRS; IBPB conditional; PBRSE-eIBRS SW sequence; BHI SW loop, KVM SW lo
op
Srbds: Not affected
Tsa: Not affected
Tsx async abort: Not affected
Vmscape: Not affected
Child process completed.
siddinapujith07@DESKTOP-FDHCAIT:~/OSSP$ ./command_exec
Enter a Linux command: lsblk

Parent Process
Parent PID: 887

Child Process
Child PID: 888
Parent PID: 887
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
sda 8:0 0 356.9M 1 disk
sdb 8:16 0 159.4M 1 disk
sdc 8:32 0 2G 0 disk [SWAP]
sdd 8:48 0 1T 0 disk /mnt/wslg/distro

```

```

Child process completed.
siddinapujith07@DESKTOP-FDHCAIT:~/OSSP$ ./command_exec
Enter a Linux command: ps

Parent Process
Parent PID: 898

Child Process
Child PID: 909
Parent PID: 898
  PID TTY          TIME CMD
  384 pts/0    00:00:00 bash
  898 pts/0    00:00:00 command_exec
  909 pts/0    00:00:00 ps
Child process completed.
siddinapujith07@DESKTOP-FDHCAIT:~/OSSP$ ./command_exec
Enter a Linux command: top

Parent Process
Parent PID: 921

Child Process
Child PID: 922
Parent PID: 921
top - 10:07:54 up 10 min, 1 user, load average: 0.05, 0.03, 0.00
Tasks: 26 total, 1 running, 25 sleeping, 0 stopped, 0 zombie
%Cpu(s):  0.0 us,  0.5 sy,  0.0 ni, 99.5 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :  7783.6 total,  7216.6 free,   557.6 used,  160.3 buff/cache
MiB Swap:  2048.0 total,  2048.0 free,    0.0 used,  7226.1 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR S  %CPU  %MEM    TIME+  COMMAND
    1 root        20   0   24564   15616 11636 S   0.0   0.2   0:02.02 systemd
    2 root        20   0    3180    2208   2072 S   0.0   0.0   0:00.02 init-systemd(Ub

```

```
top - 10:08:03 up 10 min, 1 user, load average: 0.04, 0.03, 0.00
Tasks: 26 total, 1 running, 25 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.1 us, 0.1 sy, 0.0 ni, 99.7 id, 0.0 wa, 0.0 hi, 0.1 si, 0.0 st
MiB Mem : 7783.6 total, 7222.4 free, 551.7 used, 160.3 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used, 7231.9 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	24564	15616	11636	S	0.0	0.2	0:02.02	systemd
2	root	20	0	3180	2208	2072	S	0.0	0.0	0:00.02	init-systemd(Ub
6	root	20	0	3180	2044	1940	S	0.0	0.0	0:00.00	init
52	root	19	-1	50396	16840	15552	S	0.0	0.2	0:00.32	systemd-journal
84	systemd+	20	0	22540	14676	12060	S	0.0	0.2	0:00.18	systemd-resolve
92	root	20	0	35312	12352	9312	S	0.0	0.2	0:00.82	systemd-udev
113	root	20	0	2888	2012	1884	S	0.0	0.0	0:00.10	chronyd-starter
114	root	20	0	4472	3204	2952	S	0.0	0.0	0:00.01	cron
115	message+	20	0	8816	5496	4824	S	0.0	0.1	0:00.08	dbus-daemon
137	root	20	0	44092	29848	14712	S	0.0	0.4	0:00.54	networkd-dispat
164	root	20	0	18432	9484	8236	S	0.0	0.1	0:00.23	systemd-logind
169	root	20	0	1792300	15296	12616	S	0.0	0.2	0:00.20	wsl-pro-service
194	syslog	20	0	220548	6000	4712	S	0.0	0.1	0:00.16	rsyslogd
216	root	20	0	5492	2796	2612	S	0.0	0.0	0:00.02	agetty
260	_chrony	20	0	24248	11728	7156	S	0.0	0.1	0:00.11	chronyd
274	_chrony	20	0	12092	2508	1864	S	0.0	0.0	0:00.02	chronyd
341	root	20	0	123456	32656	18044	S	0.0	0.4	0:00.34	unattended-upgr
382	root	20	0	3184	1112	980	S	0.0	0.0	0:00.00	SessionLeader
383	root	20	0	3200	1252	1108	S	0.0	0.0	0:00.13	Relay(384)
384	siddina+	20	0	6268	5600	3928	S	0.0	0.1	0:00.14	bash
385	root	20	0	8644	5628	4788	S	0.0	0.1	0:00.02	login
430	siddina+	20	0	22504	13500	11048	S	0.0	0.2	0:00.20	systemd
434	siddina+	20	0	22912	4124	2120	S	0.0	0.1	0:00.00	(sd-pam)
463	siddina+	20	0	6136	5464	3916	S	0.0	0.1	0:00.06	bash
921	siddina+	20	0	2768	1872	1776	S	0.0	0.0	0:00.00	command_exec
922	siddina+	20	0	8156	6056	3964	R	0.0	0.1	0:00.02	top

Code:

```
GNU nano 8.7.1                                command_exec.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

int main() {
    char command[100];

    printf("Enter a Linux command: ");
    fgets(command, sizeof(command), stdin);

    // Remove newline character
    command[strcspn(command, "\n")] = '\0';

    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
        return 1;
    }
    else if (pid == 0) {
        // Child Process
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        execlp(command, command, NULL);
    }

    return 0;
}
```

[^]G Help [^]O Write Out [^]F Where Is [^]K Cut [^]T Execute [^]C Location ^M-U Undo ^M-A Set Mark
[^]X Exit [^]R Read File [^]\ Replace [^]U Paste [^]J Justify [^]/ Go To Line ^M-E Redo ^M-G Copy

```
pid_t pid = fork();

if (pid < 0) {
    printf("Fork failed!\n");
    return 1;
}
else if (pid == 0) {
    // Child Process
    printf("\nChild Process\n");
    printf("Child PID: %d\n", getpid());
    printf("Parent PID: %d\n", getppid());

    execlp(command, command, NULL);

    // Executes only if exec fails
    printf("Invalid command!\n");
    exit(1);
}
else {
    // Parent Process
    printf("\nParent Process\n");
    printf("Parent PID: %d\n", getpid());
    wait(NULL);
    printf("Child process completed.\n");
    return 0;
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo	M-A Set Mark
^X Exit	^R Read File	^H Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo	M-C Copy